

Detección de amenazas con YARA y Sigma

Un ejemplo práctico para clase: fichero, memoria y logs

Universidad de Almería

Material docente

22 de septiembre de 2026

Resumen

Esta práctica muestra, con ejemplos ejecutables y salidas reales, cómo funcionan dos de las herramientas de detección más usadas en seguridad defensiva: **YARA**, que busca patrones dentro de *ficheros* y de la *memoria* de procesos en ejecución, y **Sigma**, un formato genérico de reglas para *logs* que se traduce al lenguaje de consulta de cada plataforma (Splunk, Elasticsearch, SQLite, etc.). El hilo conductor es una muestra de laboratorio inofensiva que cifra sus indicadores en disco y los descifra en memoria, lo que permite ver con claridad por qué el análisis de RAM es imprescindible.

Índice

1. Objetivos y montaje	1
2. La muestra de laboratorio	2
3. YARA sobre ficheros	3
4. YARA sobre la memoria (RAM)	4
5. Sigma: una regla, muchos backends	5
6. YARA y Sigma juntos	6
7. Reproducir la práctica	7

1. Objetivos y montaje

Al terminar la práctica el alumnado sabrá:

- Escribir reglas YARA y aplicarlas a un **fichero** en disco.
- Aplicar la **misma** regla a la **RAM** de un proceso vivo y entender por qué el resultado es distinto.
- Escribir una regla **Sigma** y traducirla a varios lenguajes de consulta, ejecutándola contra un log real.

Idea clave

El malware moderno oculta sus indicadores en disco (cifrado, empaquetado, ofuscación). Solo cuando se ejecuta se descifra en memoria. Por eso una regla que *falla* contra el fichero puede *acertar* contra la RAM del proceso. Este es el núcleo pedagógico del ejemplo.

Herramientas usadas

Herramienta	Versión	Para qué
YARA	4.5.8	Reglas sobre ficheros y memoria
yara-python	4.5.4	Uso de YARA desde Python (opcional)
sigma-cli	3.1.0	Conversión de reglas Sigma
SQLite	3.45	Log de ejemplo consultable
gcc	(sistema)	Compilar la muestra de laboratorio

Estructura de ficheros del proyecto

```

yara/
  muestras/
    muestra_demo.c      # "malware" de laboratorio (inofensivo)
    compilar.sh         # compila la muestra
    eicar.com           # fichero de prueba estandar EICAR
    informe_limpio.txt  # control negativo (fichero limpio)
  reglas/
    demo_fichero.yar    # reglas para escanear ficheros
    demo_ram.yar        # reglas para escanear memoria
  ejecutar_yara.sh      # script que ejecuta toda la demo de YARA
sigma/
  reglas/
    proceso_sospechoso.yml
    beacon_red.yml
  logs/
    eventos.sql         # log sintetico de creacion de procesos
  ejecutar_sigma.sh     # script que ejecuta toda la demo de Sigma

```

2. La muestra de laboratorio

La muestra es un programa en C **inofensivo**. No se conecta a ningún sitio ni hace nada dañino: se limita a descifrar en memoria dos indicadores y a esperar en un bucle. Los indicadores son un dominio de mando y control (C2) ficticio y un marcador de *payload*. En disco están cifrados con una operación XOR sencilla, de modo que **no aparecen en texto plano** en el binario.

```

#define CLAVE_XOR 0x5A

/* "http://c2.malicioso-demo.local/beacon" cifrado con XOR 0x5A */
static unsigned char c2_cifrado[] = { 0x32,0x2e,0x2e,0x2a, /* ... */ };

static char *descifrar(const unsigned char *src, size_t n) {
    char *out = malloc(n + 1);
    for (size_t i = 0; i < n; i++)
        out[i] = (char)(src[i] ^ CLAVE_XOR); /* descifrado en RAM */
    out[n] = '\0';
    return out;
}

```

Listing 1: Fragmento de muestra_demo.c

Nota de seguridad

La muestra llama a `prctl(PR_SET_PTRACER, PR_SET_PTRACER_ANY, ...)` únicamente para que, en la demo, **yara** pueda leer su memoria sin privilegios de administrador. En un análisis real se usaría `sudo` o un entorno aislado. La muestra no realiza ninguna acción peligrosa.

Se compila así, quitando los símbolos para que se parezca a una muestra real:

```
$ bash yara/muestras/compilar.sh
Compilado: ../yara/muestras/muestra_demo
```

Comprobamos que las cadenas comprometedoras **no** están en claro en el binario:

```
$ strings yara/muestras/muestra_demo | grep -Ei "c2|EVIL|malicioso"
(sin resultados)
```

3. YARA sobre ficheros

Una regla YARA tiene tres secciones: **meta** (información descriptiva), **strings** (los patrones a buscar) y **condition** (la expresión booleana que decide si la regla “salta”). Los patrones pueden ser texto, secuencias hexadecimales o expresiones regulares.

```
import "elf"      // inspeccionar cabeceras ELF (Linux)

rule EICAR_Fichero_Prueba : prueba {
  strings:
    $eicar = "EICAR-STANDARD-ANTIVIRUS-TEST-FILE"
  condition:
    $eicar
}

rule Malware_Demo_Disco : demo linux {
  strings:
    $s1 = "[muestra_demo]"
    $s2 = "Cadenas descifradas en memoria"
  condition:
    (elf.type == elf.ET_EXEC or elf.type == elf.ET_DYN)
    and any of ($s*)
}

rule C2_En_Claro : demo c2 {
  strings:
    $dominio = "c2.malicioso-demo.local"  ascii wide nocase
    $payload = "EVIL_PAYLOAD_DEMO_v1"     ascii wide
  condition:
    any of them
}
```

Listing 2: yara/reglas/demo_fichero.yar (extracto)

Ejecución

La sintaxis es `yara REGLAS OBJETIVO`. Escaneamos cada muestra:

```
$ yara yara/reglas/demo_fichero.yar yara/muestras/muestra_demo
Malware_Demo_Disco yara/muestras/muestra_demo

$ yara yara/reglas/demo_fichero.yar yara/muestras/eicar.com
EICAR_Fichero_Prueba yara/muestras/eicar.com

$ yara yara/reglas/demo_fichero.yar yara/muestras/informe_limpio.txt
(sin coincidencias: es el control negativo)
```

Idea clave

Fíjate en lo que **no** ha pasado: la regla **C2_En_Claro** no ha saltado contra el binario. El dominio del C2 existe en el programa, pero está cifrado en disco, así que YARA no lo encuentra. Lo veremos aparecer en la sección siguiente, al escanear la memoria.

Un detalle didáctico: si escaneamos la carpeta en modo recursivo (**yara -r**), la regla **C2_En_Claro** *sí* salta contra el **código fuente** `muestra_demo.c`, porque ahí el dominio aparece en un comentario en texto plano. El mismo indicador está oculto en el binario y visible en el fuente.

```
$ yara -r yara/reglas/demo_fichero.yar yara/muestras/
Malware_Demo_Disco yara/muestras//muestra_demo
C2_En_Claro yara/muestras//muestra_demo.c
EICAR_Fichero_Prueba yara/muestras//eicar.com
```

4. YARA sobre la memoria (RAM)

YARA puede escanear la memoria de un proceso en ejecución pasándole su **PID** en lugar de un fichero. Recorre las regiones de memoria del proceso buscando los mismos patrones. Usamos las reglas de `demo_ram.yar`, que buscan los indicadores **en claro**.

```
rule Malware_Demo_En_Memoria : demo ram {
  strings:
    $dominio = "c2.malicioso-demo.local"  ascii wide nocase
    $payload = "EVIL_PAYLOAD_DEMO_v1"     ascii wide
    $url      = "http://c2.malicioso-demo.local/beacon"  ascii
  condition:
    any of them
}

rule Beacon_C2_Generico : ram heuristica {
  strings:
    // URL de beacon http hacia un host 'c2.*'
    $beacon = /https?:\/\c2\[a-z0-9.\-]+\beacon/  ascii nocase
  condition:
    $beacon
}
```

Listing 3: `yara/reglas/demo_ram.yar`

Ejecución

Lanzamos la muestra en segundo plano, anotamos su PID, y comparamos el escaneo del fichero con el escaneo de la memoria:

```
$ ./yara/muestras/muestra_demo &
[muestra_demo] PID 93636
[muestra_demo] Cadenas descifradas en memoria. Esperando...

# 1) La regla de C2 contra el FICHERO en disco: FALLA
$ yara yara/reglas/demo_ram.yar yara/muestras/muestra_demo
(sin salida)

# 2) La MISMA regla contra la RAM del proceso: ACIERTA
$ yara -s yara/reglas/demo_ram.yar 93636
Malware_Demo_En_Memoria 93636
0x60278e3202a7:$dominio: c2.malicioso-demo.local
0x60278e3202d0:$payload: EVIL_PAYLOAD_DEMO_v1
0x60278e3202a0:$url: http://c2.malicioso-demo.local/beacon
Beacon_C2_Generico 93636
0x60278e3202a0:$beacon: http://c2.malicioso-demo.local/beacon
```

La opción `-s` muestra, para cada coincidencia, la **dirección de memoria** y el **contenido** encontrado. Vemos los tres indicadores ya descifrados, exactamente donde el malware los dejó en la RAM.

Idea clave

Mismo binario, misma regla, resultado opuesto. En disco los indicadores están cifrados y la regla falla; en memoria están descifrados y la regla los encuentra. Esta es la razón por la que el *análisis de memoria* (RAM forensics) es una técnica central en la respuesta ante incidentes.

Del PID al volcado de memoria completo

En un caso real no siempre hay un proceso vivo que escanear. Lo habitual es capturar un **volcado de memoria** de toda la máquina y analizarlo en frío, por ejemplo con **Volatility**. YARA se integra en ese flujo: se puede escanear el volcado o los procesos extraídos de él con las mismas reglas. El principio es idéntico al de esta práctica; solo cambia el origen de los bytes.

5. Sigma: una regla, muchos backends

YARA mira *bytes* (archivos y memoria). Sigma mira *logs*. Una regla Sigma se escribe una sola vez en YAML, de forma independiente de la plataforma, y luego se **traduce** al lenguaje de consulta concreto: SPL para Splunk, Lucene o EQL para Elasticsearch, SQL para SQLite, etc. Así una misma lógica de detección se comparte entre organizaciones que usan herramientas distintas.

```
title: Ejecucion sospechosa con conexion a C2 de demostracion
status: experimental
logsource:
  category: process_creation
  product: linux
detection:
  selection_imagen:
    Image|endswith: '/muestra_demo'
  selection_c2:
    CommandLine|contains: 'c2.malicioso-demo.local'
  condition: selection_imagen or selection_c2
fields: [Image, CommandLine, User]
level: high
tags: [attack.command-and-control, attack.t1071.001]
```

Listing 4: sigma/reglas/proceso_sospechoso.yml

Traducción a varios lenguajes

Con `sigma convert -t BACKEND` obtenemos la consulta en cada plataforma. La **misma** regla produce:

```
$ sigma convert -t splunk -without-pipeline proceso_sospechoso.yml
Image="*/muestra_demo" OR CommandLine="*c2.malicioso-demo.local*"
| table Image,CommandLine,User

$ sigma convert -t lucene -without-pipeline proceso_sospechoso.yml
Image:*/muestra_demo OR CommandLine:*c2.malicioso\demo.local*

$ sigma convert -t sqlite proceso_sospechoso.yml
SELECT * FROM <TABLE_NAME> WHERE Image LIKE '%/muestra_demo' ESCAPE '\ '
OR CommandLine LIKE '%c2.malicioso-demo.local%' ESCAPE '\ '
```

Ejecución real contra un log

Para que no sea solo teoría, cargamos un log sintético de creación de procesos en una base SQLite y ejecutamos la consulta que Sigma ha generado. El log mezcla actividad normal con dos eventos maliciosos.

```
$ sqlite3 eventos.db < eventos.sql
$ CONSULTA=$(sigma convert -t sqlite proceso_sospechoso.yml | sed 's/<TABLE_NAME>/eventos/')
$ sqlite3 -header -column eventos.db "$CONSULTA"
```

El resultado detecta exactamente los dos eventos maliciosos y **descarta el ruido** (bash, curl a la web de la universidad, python):

UtcTime	Image	CommandLine
2026-09-22 08:02:11	.../muestra_demo	./muestra_demo
2026-09-22 08:02:30	/usr/bin/curl	curl http://c2.malicioso-demo.local/ beacon

El primer evento coincide por el *nombre de la imagen* (`selection_imagen`); el segundo, por la *línea de comandos* (`selection_c2`). Los tres eventos benignos no aparecen.

6. YARA y Sigma juntos

Las dos herramientas son complementarias, no rivales. Cubren capas distintas de la detección:

	YARA	Sigma
Qué inspecciona	Ficheros y memoria (bytes)	Logs y eventos
Dónde actúa	Endpoint, análisis forense, sandbox	SIEM, EDR, plataforma de logs
Formato	Reglas <code>.yar</code> propias	YAML genérico traducible
Pregunta que responde	¿Este artefacto contiene el patrón?	¿Ha ocurrido este evento?

Idea clave

Un flujo de detección realista los encadena: Sigma alerta en el SIEM de que un proceso sospechoso se ha ejecutado (evento de log); a raíz de esa alerta, el analista lanza YARA contra la memoria de ese proceso para confirmar la infección (bytes). Log → alerta → confirmación en memoria.

7. Reproducir la práctica

Todo el ejemplo se ejecuta con dos scripts, desde la raíz del proyecto:

```
# Demo completa de YARA (fichero + memoria)
$ bash yara/ejecutar_yara.sh

# Demo completa de Sigma (conversion + consulta real)
$ bash sigma/ejecutar_sigma.sh
```

Ejercicios propuestos

1. Cambia la clave XOR de la muestra y comprueba que las reglas de RAM siguen funcionando sin tocarlas, mientras que buscar el texto cifrado en disco no sirve de nada.
2. Añade a `demo_ram.yar` una regla que exija que aparezcan *a la vez* el dominio y el payload (`$dominio and $payload`).
3. Escribe una regla Sigma para la categoría `network_connection` (ya tienes `beacon_red.yml` como modelo) y tradúcela a Splunk.
4. Convierte la regla Sigma a otro backend disponible (`sigma list targets`) y compara la sintaxis.

Material docente. Las muestras son de laboratorio y completamente inofensivas. El dominio `c2.malicioso-demo.local` es ficticio y no resuelve.